Елисеева Е.Р. 090303-ПИа-о24

ЛАБОРАТОРНАЯ РАБОТА №5
Тема: Функциональные возможности Python: функции высшего порядка, итераторы и генераторы
Вариант №5: Каталог фильмов

1. Цель работы

Получить практические навыки использования функциональных возможностей языка Python для обработки коллекций данных и построения ленивых последовательностей.

2. Постановка задачи

Разработать систему обработки списка фильмов. Для каждого фильма хранить: название, жанр, год выпуска, рейтинг, продолжительность. Программа должна демонстрировать использование функциональных возможностей языка Python: функций высшего порядка, анонимных функций lambda, встроенных инструментов map(), filter(), sorted(), а также проверок any(). Программа должна включать собственный класс-итератор, ленивый генератор через оператор yield и наглядное сравнение эффективности расхода памяти обычного списка и генераторного выражения.

3. Краткие теоретические сведения

В языке Python функции являются объектами первого класса, что позволяет передавать их в качестве параметров в другие функции, называемые функциями высшего порядка. Для реализации компактных локальных условий применяются анонимные функции lambda. Встроенная функция filter() отбирает элементы по заданному предикату, а map() трансформирует элементы коллекции, при этом оба инструмента возвращают итераторы, вычисляя значения по требованию.

Ленивые вычисления позволяют экономить оперативную память компьютера за счет генерации элементов по одному в момент обращения. Функция-генератор использует ключевое слово yield, которое возвращает значение, но сохраняет состояние функции для следующего вызова. Протокол итератора в Python требует реализации магических методов __iter__() и __next__(). При исчерпании элементов метод __next__() возбуждает стандартное исключение StopIteration, которое автоматически обрабатывается циклами for.

4. Исходный код программы

Файл: models.py

```
class Movie:

    def __init__(self, name, genre, year, rating, duration):
        self.name = name
        self.genre = genre
        self.year = year
        self.rating = rating
        self.duration = duration

    def __str__(self):
        return f"'{self.name}' [{self.genre}] | Год: {self.year}
| Рейтинг: {self.rating} | Время: {self.duration} мин."  #
возвращаем оформленную строку
```

Файл: generators.py

```
class MovieIterator:    # собственный класс-итератор

    def __init__(self, movie_list): # constructor, принимает
исходный список фильмов
        self.movies = movie_list # сохраняем список фильмов
внутри объекта
        self.index = 0 # счетчик (индекс) для отслеживания
текущего элемента

    def __iter__(self): # метод, который делает объект итератором
        return self # возвращает сам объект-итератор

    def __next__(self): # метод для получения следующего элемента
коллекции
        if self.index >= len(self.movies): # проверка, если
индекс дошел до конца списка
            raise StopIteration  # сигнал, что элементы
закончились
        result = self.movies[self.index] # берем текущий фильм из
списка по индексу
        self.index += 1 # увеличиваем индекс на 1 для следующего
шага
```

```
            return result # возвращаем найденный фильм наружу


        def get_movies_by_year_generator(movie_list, target_year):  #
        ленивый генератор фильмов определенного года через yield
            for m in movie_list: # бежим циклом по всем фильмам в списке
                if m.year == target_year: # если год фильма совпал с
        искомым годом
                    yield m  # возвращает фильм по одному, экономия
        память компьютера
```

Файл: operations.py

```
        def filter_movies(movie_list, predicate_func): # функция высшего
        порядка для фильтрации
            return filter(predicate_func, movie_list) # отбирает фильмы
        по переданной функции-условию


        def transform_movies(movie_list, transform_func): # функция
        высшего порядка для преобразования
            return map(transform_func, movie_list) # превращает объекты
        фильмов в новые данные (например, имена)


        def sort_movies(movie_list, key_func): # функция высшего порядка
        для сортировки
            return sorted(movie_list, key=key_func) # сортирует список по
        правилу из переданной функции
```

Файл: main.py

```
        from generators import MovieIterator,
        get_movies_by_year_generator
        from models import Movie
        import operations
        import sys  # для оценки эффективности генераторов по памяти


        def get_num_input(prompt, is_float=False):  # твоя функция
        безопасного ввода чисел
            while True:
                try:
                    return float(input(prompt)) if is_float else
        int(input(prompt))
                except ValueError:
                    print("ошибка. введите числовое значение!")


        def main():  # главная функция
            movies = [  # базовый список со стартовыми фильмами
                Movie("Интерстеллар", "Фантастика", 2014, 8.6, 169),
                Movie("Начало", "Фантастика", 2010, 8.6, 148),
                Movie("Зеленая миля", "Драма", 1999, 9.1, 189),
                Movie("Побег из Шоушенка", "Драма", 1994, 9.1, 142),
                Movie("Джентльмены", "Комедия", 2019, 8.5, 113),
            ]

            while True:  # цикл интерактивного меню
```

```
        print("\n===== МЕНЮ ФУНКЦИОНАЛЬНОГО АНАЛИЗА (ЛАБА 5)
=====")
        print("1. показать все фильмы | 2. фильтр по рейтингу |
3. только названия | 4. сорт по длительности")
        print("5. средняя длительность | 6. проверить жанр | 7.
ленивый генератор года | 8. тест памяти | 0. выход")

        choice = input("выберите действие: ").strip()

        if choice == "1":
            print("\nсписок всех фильмов в базе:")
            iterator = MovieIterator(movies)  # создаем экземпляр
нашего итератора
            for m in iterator:  # перебираем элементы через
итератор
                print(f"- {m}")

        elif choice == "2":
            r_i = get_num_input("введите минимальный рейтинг: ",
is_float=True)
            filtered = operations.filter_movies(movies, lambda x:
x.rating >= r_i)  # вызываем filter с лямбдой
            result = list(filtered)  # превращаем итератор в
список для вывода
            if result:
                print(f"\nфильмы с рейтингом >= {r_i}:")
                for m in result:
                    print(f"- {m}")
            else:
                print("нет таких фильмов.")

        elif choice == "3":
            names_map = operations.transform_movies(movies,
lambda x: x.name)  # вызываем map с лямбдой
            names_list = [name for name in names_map]  # list
comprehension для сбора данных
            print("\nтолько названия всех фильмов в базе:")
            for name in names_list:
                print(f"- {name}")

        elif choice == "4":
            sorted_list = operations.sort_movies(movies, lambda
x: x.duration)  # сортировка по ключу длительности
            print("\nфильмы, отсортированные по возрастанию
длительности:")
            for m in sorted_list:
                print(f"- {m.name} ({m.duration} мин.)")

        elif choice == "5":
            if not movies:
                continue
            durations = [m.duration for m in movies]  # собираем
длительности через comprehension
            avg_duration = round(sum(durations) / len(movies), 2)
            print(f"\nсредняя продолжительность фильмов:
{avg_duration} мин.")

        elif choice == "6":
```

```
            genre_i = input("какой жанр проверить?:
").strip().lower()
            has_genre = any(m.genre.lower() == genre_i for m in
movies)  # проверка через встроенный any()
            if has_genre:
                print(f"да! в базе есть фильмы с жанром
'{genre_i}'")
            else:
                print(f"нет, фильмов жанра '{genre_i}' в базе не
обнаружено")

        elif choice == "7":
            year_i = get_num_input("фильмы какого года выдать?:
")
            print(f"\nзапуск генератора для {year_i} года:")
            gen = get_movies_by_year_generator(movies, year_i)  #
получаем генератор

            found = False
            while True:
                try:
                    m = next(gen)  # вручную извлекаем элемент
через next()
                    print(f"Генератор выдал объект: {m}")
                    found = True
                except StopIteration:  # отлавливаем завершение
работы генератора
                    print("генератор успешно завершил выдачу
данных.")
                    break
            if not found:
                print("в этом году фильмов нет.")

        elif choice == "8":
            print("\n--- ДЕМОНСТРАЦИЯ ЛЕНИВЫХ ВЫЧИСЛЕНИЙ ---")
            limit = 1000000
            list_res = [x * x for x in range(limit)]  # обычный
список в памяти
            gen_res = (x * x for x in range(limit))  # ленивое
генераторное выражение
            print(f"размер обычного списка в памяти:
{sys.getsizeof(list_res)} байт")
            print(f"размер ленивого генератора в памяти:
{sys.getsizeof(gen_res)} байт")
            print("вывод: генератор занимает в разы меньше
памяти, так как считает на лету!")

        elif choice == "0":
            print("работа программы завершена.")
            break


if __name__ == "__main__":
    main()
```

5. Результаты тестирования программы

Тест 1. Вывод всей коллекции данных на экран консоли с использованием созданного пользовательского класса-итератора MovieIterator продемонстрирован на рисунке 5.1.

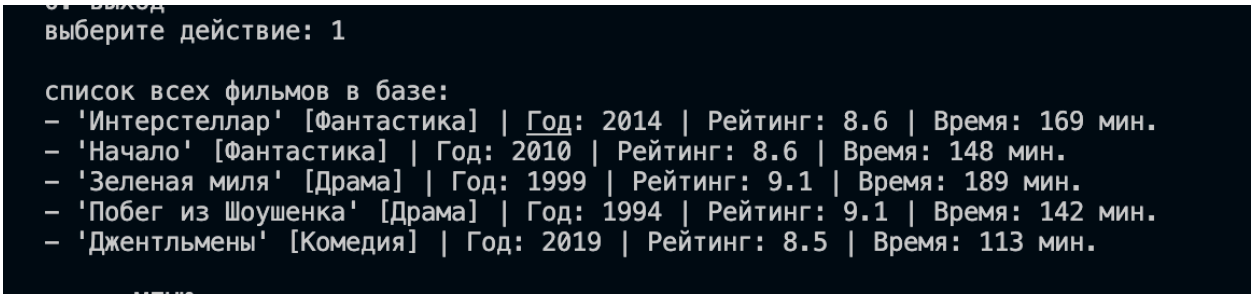


Рисунок 5.1 - класс-итератор MovieIterator

Тест 2. Фильтрация фильмов по заданному критерию рейтинга с применением анонимной lambda-функции и встроенного метода filter() показана на рисунке 5.2.

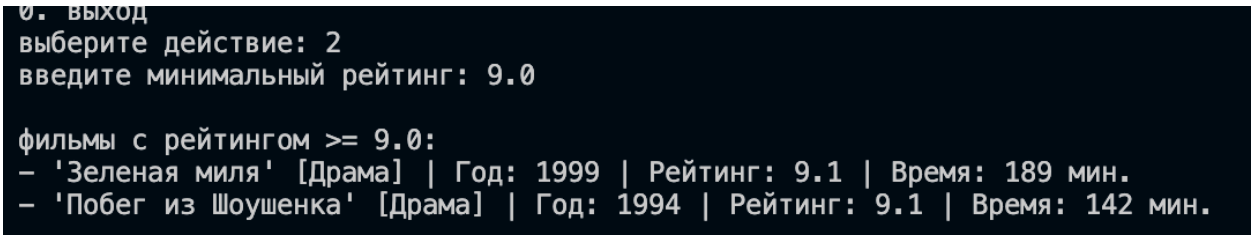


Рисунок 5.2 - фильтрация фильмов

Тест 3. Преобразование структуры данных с помощью map() для извлечения плоского списка строковых названий всех фильмов (см. Рисунок 5.3).

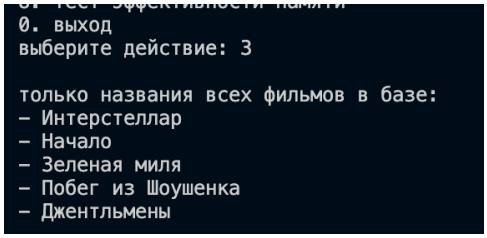


Рисунок 5.3 - названия всех фильмов, внесенных в базу

Тест 4. Пошаговая работа ленивой функции-генератора с оператором yield, ручной вызов элементов через next() и успешный перехват исключения StopIteration приведена на рисунке 5.4.

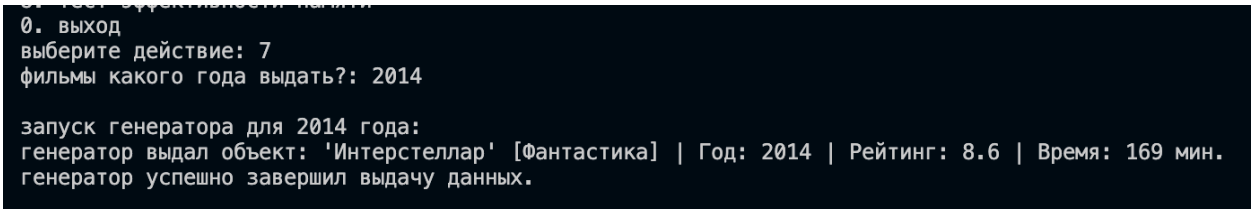


Рисунок 5.4 - работа функции-генератора

Тест 5. Сравнительный анализ потребления оперативной памяти компьютера обычным списком и ленивым генераторным выражением через модуль sys показан на рисунке 5.5.

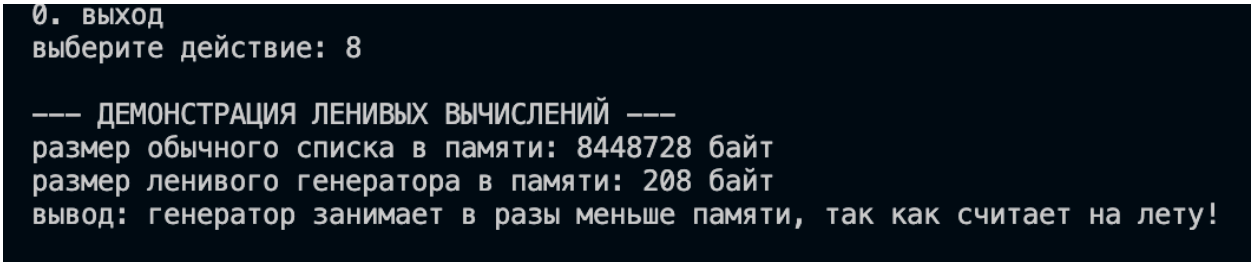


Рисунок 5.5 - демонстрация ленивых вычислений

6. Вывод

В процессе выполнения лабораторной работы были успешно освоены функциональные возможности языка программирования Python. Были изучены механизмы работы функций высшего порядка, принимающих другие функции в качестве аргументов, а также применение анонимных выражений lambda для локальной обработки структур. Научились использовать встроенные ленивые инструменты map() и filter(). На практике освоили протокол итератора при написании собственного класса MovieIterator с поддержкой методов __iter__ и __next__. Доказали высокую эффективность

ленивых вычислений и генераторов с ключевым словом yield по сравнению со стандартными списками в задачах оптимизации использования оперативной памяти приложения.